Project 2: Heart Disease Classification Report

Initial Data Exploration (EDA)

- Before beginning the cleaning process, we conducted an initial exploration which revealed that the dataset contained 920 samples across 16 initial features.
- We identified several challenges, including a target variable ('num') that was not initially binary, as well as significant missing data in columns like 'ca' (611 missing), 'slope' (309 missing), and 'thal' (486 missing).
- A correlation matrix of the raw data showed that some features had weak linear relationships with the target, while others existing on vastly different scales required standardization.

Data Preprocessing and Cleaning

- We started by cleaning the dataset to make it suitable for a binary classifier. The most important step was binarizing the target variable 'num'. Since the original data categorized heart disease into five different stages, we converted everything greater than 0 into a '1' to represent the presence of disease.
- Next, we removed the 'id' and 'dataset' columns. We did this because administrative information like patient IDs and hospital names doesn't help predict heart disease and would only add noise to our model. We also addressed a small gap in the 'restecg' column by dropping the two rows where that data was missing because its only two.
- To handle categorical data, we used One-Hot Encoding for features like sex, chest pain type (cp), and resting electrocardiogram results (restecg). We were careful to use "drop_first=True" to prevent multicollinearity issues that can mess up a Logistic Regression model.
- For the ordinal features like 'slope' and 'thal', we temporarily used Label Encoding so we could run our missing value imputer, and then converted them into dummy variables afterward.

- One of the biggest challenges was the missing medical data in the 'ca', 'thal' and 'slope' columns. We used a KNN Imputer with five neighbors to estimate these values. After that, we rounded the results for the label encoded columns, then mapped them back to their original names and used one-hot encoding on them to ensure the model could interpret them as categorical features.
- Finally, we standardized all features using StandardScaler so that variables with large numbers wouldn't over-influence the model's math.

Internal Model Comparison

To find the best possible version of the model, we compared two different regularization strategies. We first trained a baseline Logistic Regression model using L2 regularization (Ridge). This approach worked well, achieving an accuracy of 80.4% and an F1-Score of 0.838. This became our standard for comparison.

We then tested a second model using L1 regularization (Lasso). While L1 is often useful because it can automatically zero out less important features, it actually performed worse on this specific dataset. Its accuracy dropped to 77.2% with an F1-Score of 0.806. This told us that keeping all the features and just shrinking their coefficients slightly with L2 was the better move for medical accuracy.

Interpretation and Results Looking at the coefficients of our best model, we found that 'oldpeak' and 'sex_Male' were among the strongest predictors of heart disease. To see how well the model actually worked, we plotted an ROC Curve and calculated the Area Under the Curve (AUC). The model achieved an AUC of 0.87, which proves it is very reliable at distinguishing between patients who have heart disease and those who do not. We also utilized a Confusion Matrix to visualize the true positives and false negatives, confirming the model's clinical utility.

Conclusion After testing different iterations, the L2-Regularized Logistic Regression model is clearly the winner. It provides the best balance between precision and recall, making it a robust tool for clinical heart disease classification. The preprocessing steps, particularly the KNN imputation and binarization of the target, were essential in achieving this level of diagnostic accuracy.